SUPABASE + WEBSITE TEST CHECKLIST

Iron Grind Fitness Gym | Step-by-step verification guide

Before you begin

Use a test account and test data where possible. Run SQL in Supabase Dashboard > SQL Editor. These checks are mostly read-only; the expired-member demo insert is called out separately. Keep the browser Developer Tools Console and Network tabs available when testing.

1. Confirm the expected tables exist

Run this query. Confirm the app tables appear, especially profiles, members, membership_applications, attendance, products, sales, sale_items, and membership_history.

```
select table_name
from information_schema.tables
where table_schema = 'public'
  and table_type = 'BASE TABLE'
order by table_name;
```

2. Inspect expected columns

This helps diagnose errors such as a missing column in the Supabase schema cache.

```
select table_name, column_name, data_type, is_nullable
from information_schema.columns
where table_schema = 'public'
  and table_name in (
    'profiles', 'members', 'membership_applications', 'attendance',
    'products', 'sales', 'sale_items', 'membership_history'
  )
order by table_name, ordinal_position;
```

If a column exists here but the website says it cannot find the column, refresh PostgREST's schema cache:

notify pgrst, 'reload schema';

3. Check row-level security and policies

RLS should be enabled on tables exposed to website clients. Policies determine what signed-in members and admins can read or change.

```
select c.relname as table_name, c.relrowsecurity as rls_enabled
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r'
order by c.relname;
```

List the policies:

```
select tablename, policyname, cmd, roles, qual, with_check
from pg_policies
where schemaname = 'public'
order by tablename, policyname;
```

An empty result from a data query may simply mean there are no rows visible to the current role. SQL Editor access can differ from website access.

4. Check constraints and indexes

Use these when inserts fail with unique, foreign-key, or check-constraint errors.

```
select tablename, indexname, indexdef
from pg_indexes
where schemaname = 'public'
order by tablename, indexname;
```

```
select table_name, constraint_name, constraint_type
from information_schema.table_constraints
where table_schema = 'public'
order by table_name, constraint_name;
```

5. Verify Realtime publication

For live pending application and attendance updates, confirm these tables appear in the publication result.

```
select schemaname, tablename
from pg_publication_tables
where pubname = 'supabase_realtime'
order by schemaname, tablename;
```

Look for public.membership_applications and public.attendance.

6. Check current records

These counts show whether data has reached each table. Zero can be expected before that feature has been used.

```
select count(*) as profiles from public.profiles;
select count(*) as members from public.members;
select count(*) as applications from public.membership_applications;
select count(*) as pending_applications from public.membership_applications where status = 'pending';
select count(*) as attendance from public.attendance;
select count(*) as products from public.products;
select count(*) as sales from public.sales;
select count(*) as sale_items from public.sale_items;
select count(*) as membership_history from public.membership_history;
```

7. Inspect recent rows

Recent applications:

```
select id, application_type, email, status, submitted_at
from public.membership_applications
order by submitted_at desc limit 20;
```

Recent attendance:

```
select id, member_id, time_in, time_out, recorded_by
from public.attendance order by time_in desc limit 20;
```

Recent sales:

```
select id, receipt_number, customer_type, customer_name, total, status, sold_at
from public.sales order by sold_at desc limit 20;
```

Recent membership history:

```
select id, member_id, event_type, previous_expiry_date, new_expiry_date, recorded_at
from public.membership_history order by recorded_at desc limit 20;
```

WEBSITE TEST FLOW

Follow this order so a failure is easier to isolate.

A. Login and connectivity

1. Open the website and sign in as an admin. Confirm the dashboard loads without an error.
2. Sign in separately as a member and confirm their profile or membership loads.
3. If something fails, open browser Developer Tools (F12), then inspect Console and Network. Record the exact failed request and error.

B. New registration and approval

1. Submit a registration using a unique test email.
2. Confirm the site reports that the application was submitted.
3. Run the recent applications query. Confirm a registration row is pending.
4. In the admin dashboard, confirm it appears under Pending Applications. If practical, check whether another already-open admin tab updates without refresh.
5. Approve it. Confirm the application status changes and a member record is created.

C. Membership renewal

1. Use a member whose expiry date is within the allowed five-day renewal window.
2. Submit a renewal request from that member account.

D. QR attendance check-in and check-out

1. Open the member QR code and scan it using the staff attendance screen.
2. Confirm a row is created in attendance with time_out empty.
3. Scan the same member again to check out. Confirm that row now has time_out filled.
4. Check the admin attendance page and the member attendance view. If both are open, verify updates appear without refreshing.

E. Expired membership archive

1. Use only a safe test database. Run supabase/demo_expired_member.sql once in SQL Editor to add the clearly labeled expired fixture.
2. Open or reload the admin dashboard so its archive process runs.
3. In Reports, check Archives. Also inspect recent membership_history rows.
4. The demo member is test data and remains in the system under the archive-only workflow. Do not delete it directly if the app is designed to archive records.

F. Inventory and sales

1. Add a clearly labeled test product in the admin inventory page. Confirm it appears in the product list and public.products.
2. Record a small test sale. Confirm a sales row and related sale_items rows appear and the sale is in Reports.
3. For merchandise, confirm stock decreases. Supplements are sold by gram and do not use stock counts.

When a step fails

Write down the step, exact on-screen error, and any relevant browser Network response. Compare with the SQL checks above.

- Missing column / schema cache: inspect columns, then reload schema cache.
- Permission denied / empty website data: inspect RLS policies and test with the right account.
- Check constraint: compare the app value to the constraint definition.
- No row in database: confirm the website request succeeded and targeted the expected Supabase project.
- Live update missing: check publication membership and the browser console.

Safety note

Do not run schema migrations or data-changing demo SQL against production unless you intend to change that project. The table, column, policy, index, constraint, Realtime, and recent-row queries above are read-only; demo_expired_member.sql inserts a test member.